1. What is a Destructor in Python?

A **Destructor** is a special method that **automatically executes when an object is deleted** or goes **out of scope**.

In Python, the destructor method is:

```
def __del__(self):
    # cleanup code
```

💡 It is called when the **object is about to be destroyed** (just before memory is released).

🧱 **2. Basic Example**

```
class Demo:
    def __init__(self):
        print("Constructor: Object created")

    def __del__(self):
        print("Destructor: Object destroyed")

obj = Demo()
del obj
```

📜 **Output:**

```
Constructor: Object created
Destructor: Object destroyed
```

✅ Here:

- __init__() → runs when the object is created
- __del__() → runs when the object is deleted using del

⚙️ **3. When Does Python Call __del__() Automatically?**

Python automatically calls the destructor when:

- An object's **reference count** becomes **zero**
  (i.e., no variable refers to the object anymore)
- Or, when the **program ends**

Example 👇

```
class Example:
    def __init__(self, name):
        self.name = name
        print(f"Object {self.name} created")

    def __del__(self):
        print(f"Object {self.name} destroyed")

obj1 = Example("A")
obj2 = obj1  # another reference to same object
del obj1     # not destroyed yet because obj2 still refers
print("Deleting obj2...")
del obj2
```

🗒️ Output:

```
Object A created
Deleting obj2...
Object A destroyed
```

✅ The object is destroyed **only after all references are gone**.

💡 **4. Destructor with Local Scope**

When an object is created **inside a function**,
its destructor runs automatically when the function finishes —
because the local variable is deleted from memory.

```
class Test:
    def __init__(self):
        print("Object created")

    def __del__(self):
        print("Object destroyed")

def create_object():
    print("Inside function")
    obj = Test()

create_object()
print("Function ended")
```

📜 Output:

```
Inside function
Object created
Object destroyed
Function ended
```

✅ The object is destroyed right after the function ends (no longer needed).

⚠️ 5. Important Notes about __del__()

1. Python uses **Garbage Collection** —
   so you cannot always predict *exactly when* __del__() will run.
2. It runs **when memory is being reclaimed**, not necessarily immediately after del.

3. Avoid relying on it for **critical cleanup tasks** —
   instead, use **context managers** (with statement).

🧰 6. Real-Life Example — File Handling Cleanup

```
class FileHandler:
    def __init__(self, filename):
        self.file = open(filename, 'w')
        print("File opened")

    def write_data(self, data):
        self.file.write(data)

    def __del__(self):
        self.file.close()
        print("File closed")

fh = FileHandler("demo.txt")
fh.write_data("Hello, world!")
del fh
```

📜 Output:

```
File opened
File closed
```

✅ The destructor ensures the file is properly closed when the object is deleted.

🧩 7. Constructor vs Destructor

Feature	Constructor (__init__())	Destructor (__del__())
Purpose	Initialize object	Clean up object
Called	When object is created	When object is destroyed
Keyword	def __init__(self)	def __del__(self)
Called Automatically	✅ Yes	✅ Yes
Common Uses	Setting initial values	Releasing resources (files, DB, memory)

🧠 8. Destructor with Inheritance

When a class inherits another,
both destructors can be called (in reverse order of creation).

```
class Parent:
    def __del__(self):
        print("Parent destructor called")


class Child(Parent):
    def __del__(self):
        print("Child destructor called")
        super().__del__()


obj = Child()
del obj
```

📜 Output:

```
Child destructor called
Parent destructor called
```

✅ *The child's destructor runs first, then the parent's.*

🧪 9. Practice Exercise

Try this yourself 👇

```
# Exercise: Destructor Practice

# 1. Create a class 'DatabaseConnection'

#    - Constructor: print "Database connected"

#    - Destructor: print "Database disconnected"

# 2. Create an object inside a function and observe when the destructor is called.

# 3. Then create another object globally and delete it using 'del'.
```

📜 10. Summary Table

Concept	Description	Example
__del__()	Destructor method	def __del__(self):
Called When	Object deleted or reference count = 0	del obj
Purpose	Release resources / cleanup	Closing files, DB connections
Triggered Automatically	✅ Yes	
Order	Reverse of constructor calls	Child → Parent

🎓 Instructor Tip

Think of it like this:

📐 __init__() → builds the object
🧹 __del__() → cleans up after it's done

But — **don't depend too heavily** on destructors for resource cleanup.
For predictable cleanup, use **context managers** (with open(...) as f:).